

Gmail Query Engine

A reusable TypeScript library for safe Gmail querying, MIME extraction, and explicit message actions

Project context: Reusable backend component designed for CLI, desktop, server, or workflow hosts

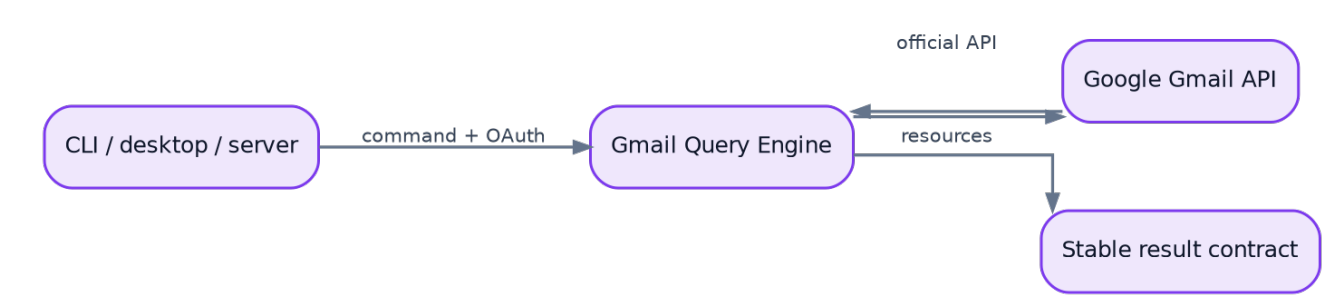
Status: Implemented library

A stateless library that validates commands, builds Gmail searches, fetches and normalizes messages, decodes MIME bodies, extracts links, and performs explicit label or compose operations without accepting Gmail passwords or owning credential storage.

Technology stack

TypeScript	Node.js	Google Gmail API	googleapis
Zod	MIME / base64url	ESM	Automated tests

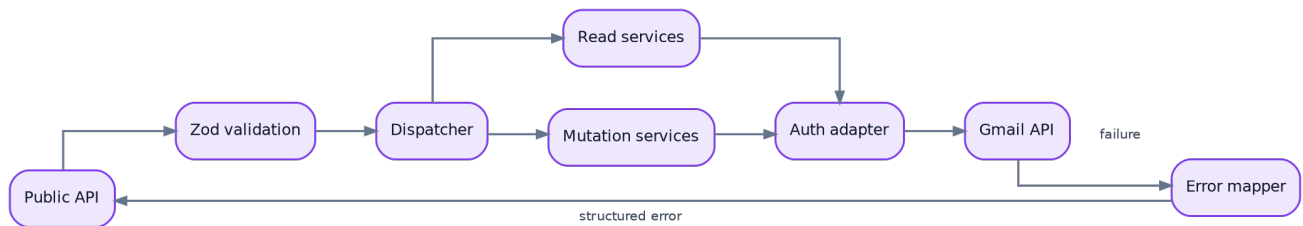
System at a glance



Search and pagination Supports raw Gmail syntax and structured filter construction.	Content extraction Normalizes metadata, plain text, HTML, attachment metadata, and links.
Explicit mutations Marks read or unread, archives, modifies labels, drafts, and sends.	Operational safeguards Bounds concurrency, body size, link count, timeout, and safe-read retries.

System design and component ownership

The public facade is separated from the Google adapter. Unknown caller input is validated before dispatch, vendor responses are converted into stable public contracts, and a small GmailClient interface allows credential-free tests.



Component	Responsibility
Public engine facade	Provides typed methods and a command-dispatch entry point.
Validation layer	Uses schemas to reject invalid commands, auth shapes, and limits early.
Read services	Build queries, list message IDs, fetch details, decode MIME, and extract links.
Mutation services	Apply precise label changes and compose MIME messages.
Auth adapter	Accepts a caller-provided access token or OAuth client.
Error mapper	Redacts vendor details and returns structured, retry-aware failures.

Key design decisions

- Do not persist OAuth tokens; authentication and refresh remain host responsibilities.
- Fetch metadata by default and make full MIME bodies opt-in.
- Never automatically retry mutations because the outcome may be ambiguous.
- Hide raw Gmail resources unless a trusted caller explicitly requests them.

Core execution flow

Every operation passes through validation and a stable result boundary. Read requests can use bounded concurrency and safe retries; mutation requests remain one-shot and surface uncertainty to the host.



Implementation notes

- Default message fetch concurrency is bounded rather than unbounded Promise.all.
- MIME parsing handles nested multipart structures and base64url payloads.
- Unsafe schemes are excluded during link extraction.
- Errors include a category, retryability, request ID, and optional retry-after value.
- Tests use a fake Gmail client so CI does not require real credentials.

Minimal typed use

```
const gmail = new GmailQueryEngine({
  auth: { kind: "accessToken", accessToken },
  limits: { fetchConcurrency: 5, requestTimeoutMs: 30_000 }
});

const result = await gmail.search({
  query: "from:alerts@example.com newer_than:1d",
  fetchBody: true,
  extractLinks: true
});

if (result.success && result.data?.emails[0]) {
  await gmail.markRead({ messageId: result.data.emails[0].messageId });
}
```

The caller still owns user confirmation, token refresh, persistence, and duplicate-send policy.

Engineering review and production path

Engineering strengths

- Strong input validation and stable contracts.
- Clear vendor boundary makes tests and future adapters easier.
- Thoughtful retry policy distinguishes reads from ambiguous writes.
- Explicit limits reduce memory, latency, and API-quota risk.

Current limitations

- Requires the host to obtain and refresh OAuth credentials.
- No application UI, database, or workflow state by design.
- Credentialed end-to-end smoke tests remain optional.
- Compose delivery still requires host-level confirmation and idempotency policy.

Recommended next steps



Why this is relevant to AI training work

- Shows precise contract design and validation of many edge-case combinations.
- Demonstrates evaluation of partial failures, malformed MIME, and unsafe links.
- Separates facts returned by Gmail from normalized conclusions exposed to callers.
- Provides a testable system for judging whether generated integration code is reliable.

Repository:

<https://github.com/SidheshwarSarangal/gmail-query-engine>

Scope and provenance:

Prepared from repository architecture, reliability/testing, authentication/security, and public-contract documentation, plus the implemented usage examples.

